

Classes and Encapsulation

The core idea of **Object-Oriented Programming** is **grouping related data** and the **actions** that operate on that data into single, logical units.

1. Classes vs. Objects

It is vital to understand the difference between the **"blueprint"** and the actual **"thing."**

- **The Class (The Blueprint):** A class is a template. It defines the structure of the data (e.g., a class **Point** needs an **x** and **y** coordinate) and the methods that can operate on it (e.g., **DistanceToOrigin()**). It does not hold any actual data itself.
- **The Object (The Instance):** An object is what you get when you build something using that blueprint. It contains the actual, real data. You can use one Point class to stamp out thousands of unique Point objects in memory (e.g., Point A is at 0,0, Point B is at 5,5).

2. Does Go Support OOP?

Yes, absolutely! However, **Go does not have a class keyword**. Instead of traditional classes, Go achieves the exact same object-oriented design using a combination of **Structs** (to hold the data fields) and **Methods** (functions uniquely attached to those structs). We will see exactly how this is written in the upcoming lessons!

3. Encapsulation (Protecting the Data)

Encapsulation is the practice of hiding the internal data of an object so that outside code cannot touch it directly. Instead, outside developers are forced to use the public **"Methods"** you provided to interact with the data.

- **Why hide the data?** Trust. If you expose the raw **x** and **y** coordinates of a point, another developer might try to manually scale the point but forget to update the y axis, leaving the object in a broken, inconsistent state.
- **The Solution:** By hiding **x** and **y** and providing a **DoubleDistance()** method, you guarantee the math is done perfectly every single time.

Support for Classes

As we learned, Go does not have a **class** keyword. Instead, it relies on attaching **Methods** to **Types** using a feature called a **Receiver**.

1. The Receiver (Go's Secret Weapon)

In Go, to turn a normal function into a **"Method"** that belongs to a specific type, you place a **Receiver Argument** in parentheses before the function name.

```
type MyInt int

// Normal Function
func Double(mi MyInt) int { // You have to pass the integer explicitly.
    return int(mi) * 2
}
```

```
// Method with a Receiver
func (mi MyInt) Double() int {
    return int(mi) * 2
}
```

By adding **(mi MyInt)** before the name, you are telling Go: "This Double function now officially belongs to the **MyInt** type."

2. Calling Methods (Dot Notation)

Once a method is attached to a type, you can use standard object-oriented "**dot notation**" to call it.

```
val := MyInt(3)
val.Double()
```

- **The Implicit Argument:** When you call **val.Double()**, it looks like the parentheses are empty. But behind the scenes, Go is secretly taking the **val** object on the left side of the dot and passing it into the method as the **mi** receiver variable!

3. Building a "Class" using Structs

A single integer is rarely enough data to represent a real-world object. To build something that looks and acts exactly like a traditional Class (holding multiple pieces of data), Go uses **Structs**.

- **Step 1: Define the Data (The Struct):**

```
type Point struct {
    x float64
    y float64
}
```

- **Step 2: Attach type a Method:** You create a method and set the Receiver to be your custom Point struct.

```
func (p Point) DistToOrigin() float64 {
    return math.Sqrt(p.x * p.x + p.y * p.y)
}
```

- **Step 3: Instantiate and Use (The Object):**

```
p1 := Point{x: 3, y: 4}
fmt.Println(p1.DistToOrigin()) // Outputs: 5
```

The Critical Rule: Same Package Only

You can only attach methods to a type if the type is defined in the **exact same package** as your method.

- This means you cannot just write **func (i int) Double()** to add a custom method to Go's built-in int type.

Expert Takeaway

Methods use Call by Value by default!

When you write **func (p Point) Move()**, and you call **p1.Move()**, Go creates a complete copy of p1 and hands the copy to the method. If your Move method tries to change the **x** and **y** coordinates, it will only change the copy! The original **p1** object will stay exactly where it was.